

TRABALLO FIN DE GRAO
GRAO EN CIENCIA E ENXEÑARÍA DE DATOS

Visualizando el aprendizaje máquina: de la abstracción al entendimiento

Estudiante	Héctor Aldao Amoedo
Dirección:	Alejandro Rodríguez Árias Samuel Magaz Romero

A Coruña, junio de 2026.

Dedicatoria¿?

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. ¿?

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Palabras clave:

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext
- Last itemtext
- First itemtext

Keywords:

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext
- Last itemtext
- First itemtext

Índice general

1	Introducción	1
1.1	Contexto y motivación	1
1.1.1	Definición del Problema	1
1.1.2	Objetivos	2
1.1.3	Alcance y Limitaciones	2
1.1.4	Metodología de Trabajo	2
1.1.5	Estructura de la Memoria	2
2	Estado del Arte	3
3	Herramientas y técnicas	7
3.1	Godot como motor de desarrollo	7
3.2	Lenguajes de programación empleados	9
3.2.1	GDScript como base del proyecto	9
3.2.2	Python para validación de resultados	10
3.2.3	Rust para generación de fórmulas	10
3.2.4	JavaScript para integración con el navegador	10
3.3	Control de versiones y despliegue	11
4	Metodología y planificación	12
5	Desarrollo	13
5.1	Prototipo: versión 0.1	13
5.1.1	Primeras escenas	13
5.1.2	Exportación web con GitHub Pages	14
5.2	Árbol de decisión: versión 0.2	14
5.2.1	Primera arquitectura del árbol	14
5.2.2	Reorganización de la escena y primeros elementos explicativos	17

5.2.3	Animación de la construcción del árbol	20
5.2.4	Gráficas y casos explicativos en la ventana	20
5.2.5	Primer panel de selección de datos	21
5.2.6	Carga de archivos y navegación por pasos	22
5.2.7	Primera evaluación visual del árbol	23
6	Conclusiones	25
7	Trabajo futuro	26
A	Material adicional	28
	Bibliografía	31

Índice de figuras

2.1	Sección interactiva de la página web de 3Blue1Brown.	5
2.2	Página web TensorFlow Playground.	5
5.1	Versión preliminar del menú principal.	15
5.2	Primera escena de prueba del árbol.	15
5.3	Primer menú de control del entrenamiento algorítmico, con los botones para iniciar el entrenamiento y avanzar al siguiente paso antes de traducir la interfaz y aplicar los temas visuales definitivos.	16
5.4	Panel inicial de la escena del árbol para escoger entre creación manual y creación algorítmica.	18
5.5	Interfaz de entrenamiento del árbol tras traducir los controles. A la izquierda se muestra el botón para comenzar el entrenamiento y a la derecha el estado del árbol al avanzar con el botón de siguiente paso, con temas distintos para nodos internos y hojas.	18
5.6	Primera versión de la ventana informativa integrada en la vista del árbol, todavía con un mensaje inicial que indicaba al usuario que debía comenzar el entrenamiento.	19

Índice de cuadros

Capítulo 1

Introducción

PRIMER capítulo da memoria, onde xeralmente se exporán as liñas mestras do traballo, os obxectivos, etc. Incluimos un par de exemplos de citas e de referencias

LA inteligencia artificial ...?

1.1 Contexto y motivación

Los algoritmos de aprendizaje máquina son técnicas matemáticas que escapan de las fronteras de lo que es entendible a simple vista por ¿una persona cualquiera.¿ Desde sus raíces más puramente matemáticas hasta su implementación tanto hardware como software.

Esto ya era cierto en los tiempos de los primeros algoritmos de aprendizaje ¿por refuerzo, árboles o qué?, pero con el surgir de los modelos de aprendizaje profundo, esta realidad es más palpable que nunca.

Bajo estas premisas es que se siente indispensable ¿proporcionar a la gente los materiales y métodos para que puedan comprender lo mejor posible esta tecnología cada vez más omnipresente¿. ¿Y a de más el aprendizaje es mejor si es interactivo ¿?

1.1.1 Definición del Problema

Hoy en día las principales formas de aprender cómo funcionan los modelos de inteligencia artificial (sobre todo los clásicos, pero también los modernos) es a través de los siempre confiables libros y universidad, las más ancestrales fuentes de conocimiento. Entre los formatos más actuales también encontramos los vídeos divulgativos en plataformas online como Youtube, donde la posibilidad de que todo el mundo comparta su contenido ha dado lugar a ¿muchas oportunidades de ver las interpretaciones visuales de otras personas¿.

Sin embargo ¿aún no existe ninguna herramienta pedagógica que permita interactuar en tiempo real con los algoritmos.¿ ¿Programándolos se pueden ver sus resultados, su evolución en una gráfica, pero no palpar los que está sucediendo en las entrañas de la bestia mientras

mastica el equivalente a su comida: los datos, tanto en entrenamiento como en inferencia;?
((.....))

1.1.2 Objetivos

((Crear la aplicación - paso 1, hacerla - paso 2, exportarla
Publicarla para que todo el mundo la pueda disfrutar))
((creo qe hay q se rmás concreto))

Los objetivos que se plantean en este trabajo comienzan por plantearse qué algoritmos de IA son los más interesantes para ¿explicar¿. Esta es, en última instancia, una cuestión subjetiva, por lo que en este trabajo se abordó esta decisión mirando ¿de cara¿ a dos de los mayores paradigmas de la IA: la simbólica y el deeplearning. ((no se si decir de forma pragmática y decir que tenía en mente las redes neuronales, y que los árboles se parecían lo suficiente, o poner un poco de fantasía de "los árboles, uno los primeros amigos del hombre en cuánto a lo que hacer que una máquina aprenda a jugar al ajedrez..." (lo

De ambos se decidió que los mejores candidatos para protagonizar una explicación que busque asentar los fundamentos eran las versiones más fundamentales de cada uno. Del lado de la IA simbólica está el árbol de decisión ((planteado por bla bla bla en blabal)), conocido por ser ... white box Y en el deeplearning, el ladrillo fundacional, la red neuronal ((lo mismo de histoia...))

1.1.3 Alcance y Limitaciones

1.1.4 Metodología de Trabajo

1.1.5 Estructura de la Memoria

Estado del Arte

LA inteligencia artificial y el aprendizaje automático se apoyan en algoritmos cuyo funcionamiento interno no siempre resulta fácil de visualizar. Conceptos como la selección de atributos en un árbol de decisión, el cálculo de una frontera de decisión o la actualización iterativa de los parámetros de un modelo pueden resultar abstractos cuando se estudian únicamente desde una formulación teórica. Esta dificultad se acentúa en el aprendizaje profundo, donde intervienen elementos como funciones de activación, pesos, sesgos, propagación hacia delante, retropropagación o gradientes. Por ello, se han desarrollado recursos educativos orientados a hacer más comprensible el comportamiento de estos modelos mediante explicaciones visuales y simulaciones interactivas. En el contexto de este trabajo, resultan especialmente relevantes las herramientas que permiten comprender el proceso de aprendizaje de los modelos, la relación entre sus componentes internos y la forma en la que una decisión algorítmica se transforma en un resultado observable.

Uno de los recursos divulgativos más conocidos para introducir las redes neuronales es la serie de Grant Sanderson “Neural networks” [1]. Esta serie fue publicada en el *canal de YouTube* 3Blue1Brown. Este proyecto cuenta también con una página web [2], cuya principal aportación pedagógica es la posibilidad de interactuar con una red neuronal entrenada con el conjunto de datos MNIST [3]. La herramienta permite observar en tiempo real cómo la red reacciona a los cambios en el dato de entrada, representado mediante una superficie de dibujo en la que cada píxel se corresponde con una neurona de la capa de entrada, como se aprecia en la Figura 2.1.

Este tipo de recurso resulta valioso para desarrollar intuición, ya que reduce la barrera de entrada matemática y proporciona una primera aproximación visual al funcionamiento de una red. También es adecuado para motivar y contextualizar el problema gracias a una explicación apoyada en recursos visuales claros. Su formato, en cambio, conserva un carácter fundamentalmente expositivo: en la página web, dos de las diez lecciones incorporan un componente interactivo, y dicha interacción se orienta siempre a la relación entre la entrada y la

salida de una red ya entrenada. El estudiante trabaja con una explicación limitada al conjunto MNIST y sin capacidad de experimentar de forma directa con la arquitectura del modelo. Esta restricción deja abierta la necesidad de recursos más activos, en los que sea posible analizar el efecto de cada decisión de diseño sobre el comportamiento de una red neuronal.

Frente al formato audiovisual, existen herramientas que permiten manipular redes neuronales en tiempo real. Un ejemplo es “A Neural Network Playground”, desarrollado por Daniel Smilkov y Shan Carter y publicado por TensorFlow [4]. Esta aplicación permite configurar una red neuronal sencilla desde el navegador, seleccionar conjuntos de datos sintéticos, modificar hiperparámetros y observar cómo evoluciona la frontera de decisión durante el entrenamiento. En la Figura 2.2 se puede visualizar la página web.

A diferencia de los recursos puramente expositivos, la relevancia de TensorFlow Playground reside en que conecta elementos habitualmente tratados de forma abstracta con consecuencias observables. Modificar directamente la arquitectura del modelo (número de capas y neuronas) o sus hiperparámetros (tasa de aprendizaje y regularización) altera de manera inmediata el proceso de ajuste. Este enfoque interactivo facilita que el estudiante relacione conceptos críticos como el sobreajuste, la generalización o la convergencia con resultados palpables. La herramienta se centra principalmente en la frontera de decisión y en la pérdida, sin profundizar de forma detallada en operaciones internas como el cálculo de gradientes o la propagación de errores capa a capa.

Esta misma orientación hacia la experimentación aparece en herramientas centradas en arquitecturas más específicas. “GAN Lab” permite experimentar en el navegador con redes generativas adversarias [5]. Este tipo de modelo introduce una dinámica de aprendizaje distinta a la de una red supervisada clásica, ya que enfrenta dos redes: un generador, que intenta producir muestras plausibles, y un discriminador, que intenta distinguir entre datos reales y generados. La aportación principal de GAN Lab es representar visualmente una interacción compleja como la de un par de redes adversarias. La herramienta permite observar cómo cambian las distribuciones generadas durante el entrenamiento y cómo la competencia entre generador y discriminador afecta al resultado. Desde el punto de vista educativo, esto resulta útil para explicar que existen modelos de aprendizaje profundo con objetivos y esquemas de aprendizaje diferentes.

La limitación principal de GAN Lab, junto con la falta de detalle sobre ciertas operaciones internas, es que presupone una base de conocimiento mayor que la requerida para una introducción a las redes neuronales. Para comprender correctamente una GAN es necesario haber asimilado previamente ideas como función de pérdida, optimización, representación interna o entrenamiento iterativo. En consecuencia, GAN Lab es una herramienta valiosa para una fase posterior del aprendizaje, aunque no sustituye a una aplicación centrada en los fundamentos de una red neuronal básica.

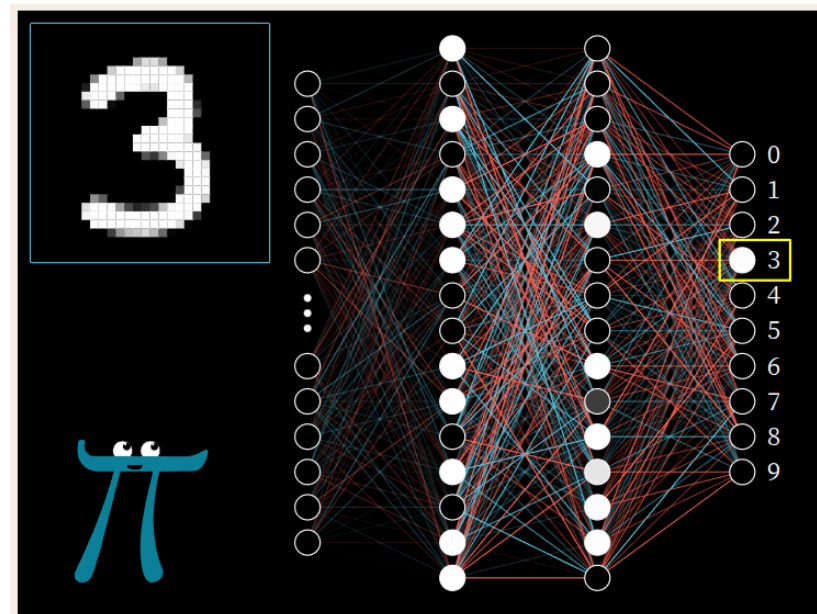


Figura 2.1: Sección interactiva de la página web de 3Blue1Brown.

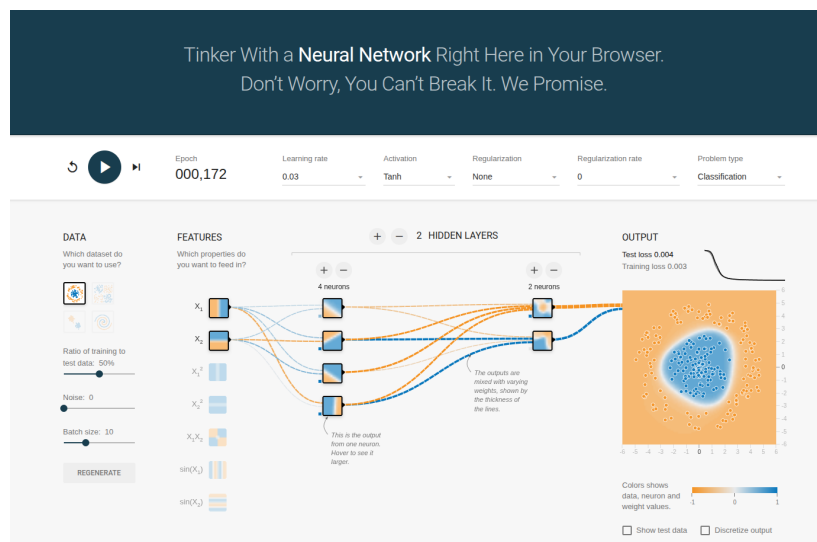


Figura 2.2: Página web TensorFlow Playground.

En el ámbito de los algoritmos clásicos de aprendizaje automático, también existen propuestas para ilustrar el funcionamiento de los árboles de decisión. La herramienta web interactiva sobre árboles de decisión de Interactive Machine Learning [6] permite al usuario construir un árbol y observar el proceso de división de un conjunto de datos en tiempo real. Esta propuesta resulta interesante porque acerca al estudiante a la estructura jerárquica del modelo y al proceso de partición sucesiva de los datos. Su experiencia de uso, no obstante, es limitada y las explicaciones que acompañan a la simulación tienen poco desarrollo.

Aunque muestra los valores de distintas métricas, no acompaña dichos valores con una interpretación suficiente de su significado ni de su papel dentro del algoritmo. Esto reduce su utilidad como recurso formativo autónomo, especialmente para estudiantes que se aproximan por primera vez a los árboles de decisión.

En comparación con las redes neuronales, la disponibilidad de herramientas interactivas centradas en árboles de decisión es más reducida. La revisión realizada muestra una presencia mucho mayor de simuladores y visualizaciones dedicados a redes neuronales, tanto en recursos divulgativos como en herramientas web orientadas a la experimentación. Esta diferencia es especialmente visible cuando se buscan recursos que combinen visualización del modelo, explicación del criterio de partición y evaluación paso a paso.

Desde la perspectiva de este proyecto, la principal oportunidad se encuentra en combinar las fortalezas de estos recursos. Una herramienta educativa eficaz debería mantener la claridad visual de los recursos divulgativos y su atención a la base matemática, incorporar la manipulación directa de los simuladores y ofrecer suficiente rigor técnico para que lo presentado al usuario se corresponda con el funcionamiento real del algoritmo. También debería evitar que la interacción se limite a modificar hiperparámetros globales. Para entender una red neuronal resulta especialmente útil poder observar el flujo de datos, la contribución de los pesos y el efecto del entrenamiento sobre cada componente del modelo. En el caso de los árboles de decisión, la visualización debería mostrar cómo se selecciona cada atributo, cómo se divide el conjunto de datos y cómo se alcanza una clasificación final a partir de las características de entrada. En ambos modelos, esta visualización debe ir acompañada de explicaciones textuales que interpreten las métricas, los cálculos y las decisiones del algoritmo, de forma que la simulación no dependa únicamente de la observación visual.

Herramientas y técnicas

EL desarrollo de este trabajo se apoya en un conjunto de herramientas orientadas a construir una aplicación educativa, interactiva y ejecutable desde el navegador. [Ahora mismo podrías hacer un ejecutable de pc sin problema, no? Sí Hay que narrar que vamos a escoger en web para accesibilidad, no por limitación](#) La elección tecnológica del entorno de desarrollo tuvo en cuenta tanto criterios de implementación como la naturaleza del problema abordado: explicar modelos de aprendizaje automático mediante visualizaciones dinámicas, animaciones y ejemplos manipulables por el usuario. En consecuencia, se priorizaron tecnologías que facilitasen la construcción de interfaces interactivas, la validación de los algoritmos implementados y el despliegue web del resultado final.

3.1 Godot como motor de desarrollo

”Libre” se entiende? La herramienta principal empleada fue Godot, un motor de desarrollo de desarrollo libre y multiplataforma orientado a la creación de aplicaciones interactivas y videojuegos [7]. [No descartaría dejarlo solo como aplicaciones interactivas y en el parrafo siguiente olvidar el concepto de videojuego, permite crear aplicaciones interactivas y creamos una aplicación interactiva \(un videojuego es una app software interactiva con un nombre fancy\) okey](#) Aunque su uso más habitual se asocia al desarrollo de videojuegos, sus características también resultan adecuadas para una aplicación educativa como la desarrollada en este trabajo. El proyecto requiere presentar información visual, responder a acciones del usuario, animar procesos paso a paso y mantener una estructura clara entre los distintos elementos de la interfaz. Estas necesidades encajan de forma natural con el modelo de escenas, nodos y señales que propone Godot [8].

A lo largo de esta memoria se hablará en repetidas ocasiones de dos de las principales estructuras con las que se trabaja en Godot: las escenas y los nodos.

Los nodos son el elemento básico y más importante con el que se trabaja en el motor. Todo

elemento y subelemento que vaya a estar presente en tiempo de ejecución en el programa es un nodo. Y cada nodo cuenta, principalmente, con una serie de atributos, métodos y señales. Igual que en la programación orientada a objetos, los atributos son variables o constantes y los métodos funciones. Las señales son disparadores de funciones. Cuando son emitidas llaman a la función a la que están conectadas. Son una forma de conectar eventos entre diferentes nodos. Son el núcleo del paradigma de programación que promueve el motor: la programación dirigida por eventos. [Aquí puse lo de la programación dirigida por eventos](#)

Los atributos, métodos y señales de cada nodo vienen definidos por su clase. Godot dispone de un gran árbol de clases predefinidas que heredan atributos y métodos de sus clases padre. Las tres clases básicas son: el “Node”, una clase vacía, la raíz del resto de nodos, que solo cuenta con los métodos mínimos para que funcione en el motor; el “Node2D”, característico por contar con atributos referentes a la posición en un espacio bidimensional; el “Node3D”, similar al anterior pero en un espacio tridimensional; y el “Control”, pensado para los elementos de interfaz de usuario. Por encima de estas, el desarrollador siempre puede definir sus propias clases ampliando las ya existentes.

Los nodos por sí solos son clases abstractas que no existen en el programa ejecutable final por sí mismos. Para instanciarlos están las escenas. Estas son la materialización concreta del proyecto: los archivos que son guardados en disco y con los que se interactúa en el editor. Están formadas, como mínimo, por un nodo raíz. De este nodo raíz pueden colgar nodos hijo, que a su vez pueden contar con sus propios nodos hijo, y así sucesivamente.

Finalmente, las escenas, como en última instancia son árboles de nodos, pueden ser instanciadas dentro de otras escenas. Esto habilita una modularidad total, puesto que toda estructura de nodos que sea susceptible de ser reutilizada, puede guardarse como una escena para no tener que ser recreada.

A mayores, los nodos pueden tener asociados un script. Estos son código desde el que se puede interactuar, principalmente, con los atributos y métodos del nodo al que están asociados, así como con sus hijos. Pero no están limitados a estos elementos. Siempre se puede acceder al nodo raíz, y desde este modificar cualquier propiedad global o acceder a cualquier nodo en la escena. O mediante la creación de “Autoloads” en el proyecto se puede acceder a clases personalizadas en cualquier punto del código.

[estaría bien tener un diagrama sencillo que explicase las relaciones escena/nodos/scripts, igual en la doc de Godot hay algo así? sí, veo si hay ya alguno. Vale encontré, he de ver dónde referenciarlo](#)

La organización en escenas permite separar componentes de la aplicación con responsabilidades distintas, como las vistas de entrenamiento, los paneles explicativos, las representaciones gráficas y los controles de interacción. Este enfoque favorece una estructura modular en la que cada parte de la interfaz puede desarrollarse, probarse y reutilizarse con menor aco-

plamiento. Es raro, al final estamos diciendo que permiten aplicar patrones de diseño En qué sentido es raro? El sistema de señales facilita la comunicación entre elementos de la aplicación sin imponer dependencias directas innecesarias, lo que resulta útil en un entorno en el que las acciones del usuario deben provocar cambios visuales y actualizaciones del estado interno de los modelos.

La manera en que Godot organiza las interfaces mediante nodos permite estructurar los elementos de los algoritmos tratados de forma sencilla. Un árbol de decisión puede mostrarse como una estructura jerárquica de nodos conectados, mientras que una red neuronal puede representarse mediante capas, neuronas y conexiones. Esta correspondencia conceptual ayuda a trasladar las estructuras internas del algoritmo a elementos visuales manipulables dentro de la aplicación.

Otro motivo importante para escoger Godot fue su capacidad de exportación a web. El motor permite generar una versión HTML5 de la aplicación [9], lo que evita exigir al usuario una instalación local y facilita el acceso desde un navegador. Esta característica es especialmente relevante en un proyecto con finalidad educativa, ya que reduce la fricción de uso y permite distribuir la herramienta como una aplicación web estática.

Yo lo que veo es que no justificamos la elección de Godot frente a herramientas de desarrollo de interfaces interactivas. ¿Que nos aporta Godot que no haga Java, .Net, Flutter etc... yo creo que se puede justificar por la parte de la interacción/animaciones, al tener el propio ciclo de vida del juego, facilita toda esa parte hmmm vale, lo hablamos

3.2 Lenguajes de programación empleados

Debatir en reunión. ¿No integraríais esto en sus propias secciones de desarrollo? rust y js sí, porque son para funcionalidades específicas cómo considereis

El desarrollo combina varios lenguajes, cada uno empleado en una parte concreta del proyecto. La lógica principal de la aplicación se implementó dentro del propio entorno de Godot, utilizando GDScript para construir las visualizaciones, gestionar la interacción y ejecutar las implementaciones didácticas de los algoritmos. Junto a ello, Python, Rust y JavaScript se emplearon como tecnologías de apoyo para resolver necesidades específicas que no formaban parte del núcleo de la aplicación.

3.2.1 GDScript como base del proyecto

GDScript es el lenguaje oficial utilizado en Godot para definir la lógica de las escenas y la interacción entre nodos. Su integración directa con el motor permitió implementar tanto el comportamiento de la interfaz como las versiones didácticas de los algoritmos de aprendizaje automático. La implementación de estos algoritmos en el propio lenguaje de la aplicación

facilitó la obtención de resultados parciales en cada paso del proceso, lo que permitió generar explicaciones detalladas y sincronizadas con la visualización.

3.2.2 Python para validación de resultados

Python se utilizó para contrastar el comportamiento de las implementaciones realizadas en la aplicación [10]. La finalidad de este uso fue disponer de un entorno externo con bibliotecas consolidadas que sirviesen como referencia para la lógica implementada en Godot. En particular, se compararon resultados producidos por la aplicación con los obtenidos mediante implementaciones estándar de árboles de decisión y redes neuronales en bibliotecas como scikit-learn [11] y PyTorch [12], respectivamente.

Este proceso ayudó a detectar errores tanto en la implementación de los algoritmos como en la conexión entre sus resultados internos y la información presentada por la interfaz. El uso concreto de Python en este proceso se detalla en el Capítulo 5.

3.2.3 Rust para generación de fórmulas

Rust se empleó para desarrollar un plugin para Godot destinado a generar fórmulas matemáticas de forma dinámica en tiempo de ejecución [13]. Esta necesidad surge porque la aplicación debe mostrar resultados numéricos y explicar los cálculos que conducen a ellos. Para ello se utilizó la biblioteca Ratex [14], que permite generar representaciones en LaTeX a partir de expresiones construidas durante la ejecución.

La elección de Rust se justifica por su rendimiento, su seguridad de memoria y su integración con Godot. En este proyecto, el complemento se compila a WebAssembly para poder utilizarlo en la versión web de la aplicación [15]. De este modo, la generación de fórmulas puede integrarse en el despliegue web sin depender de componentes nativos específicos de una plataforma. El desarrollo de este plugin y su integración con la aplicación se explican en el Capítulo 5.

3.2.4 JavaScript para integración con el navegador

JavaScript se utilizó en una parte concreta de la aplicación relacionada con la interacción con el navegador [16]. Su función principal fue permitir que el usuario cargase sus propios ficheros CSV, de forma que la herramienta no quedase limitada únicamente a conjuntos de datos predefinidos. Esta funcionalidad resulta importante desde el punto de vista educativo, porque permite experimentar con datos distintos y observar cómo cambian las decisiones del árbol o el comportamiento de la red en función de la entrada utilizada. De forma nativa, Godot permite cargar datos desde el sistema en el que se ejecuta la aplicación. En la exportación a HTML5, el funcionamiento de los navegadores modernos restringe esta característica, lo que

hizo necesaria una capa de integración específica con JavaScript. La parte concreta desarrollada en JavaScript se describe en el Capítulo 5.

3.3 Control de versiones y despliegue

Para la gestión del código fuente se utilizó Git, un sistema de control de versiones distribuido ampliamente utilizado [17]. Su uso permitió mantener un registro de los cambios y trabajar de forma incremental.

El repositorio se alojó en GitHub, lo que proporcionó almacenamiento remoto y una copia accesible del proyecto. Además, se utilizó GitHub Pages para publicar la versión web de la aplicación [18]. Este servicio permite desplegar sitios estáticos directamente desde un repositorio, por lo que encaja con la exportación web generada por Godot. La combinación de Godot y GitHub Pages permitió que el resultado final pudiese consultarse desde el navegador sin infraestructura de servidor adicional.

Yo esto lo vendería algo más. Entiendo que luego va en su cap de desarrollo ídem: aunque sea todo con github, el VCS es una cosa más trivial pero el despliegue es algo más interesante. Es que poco más hay tbh. Podría entrar en mayor profundidad en el export, ns. Pero sí, tenía pensado en entrar en algo más de detalle en la parte del plugin de Rust porque ahí tuve que hacer movidas.

Metodología y planificación

Capítulo 5

Desarrollo

En este capítulo se describe el proceso seguido para construir la aplicación. La explicación se plantea como una evolución por versiones: primero se comprobó que la idea era viable, después se desarrolló el árbol de decisión y, a partir de las primeras pruebas, se fueron incorporando explicaciones, animaciones y formas más claras de elegir los datos de entrada.

5.1 Prototipo: versión 0.1

El primer objetivo consistió en comprobar que la herramienta elegida servía para crear una aplicación interactiva y publicarla como página web. Antes de invertir más tiempo en el desarrollo, era necesario verificar tres aspectos básicos: que Godot permitía crear una interfaz sencilla, que se podía representar una estructura con forma de árbol y que el resultado podía ejecutarse desde el navegador.

Por ese motivo, la primera versión del proyecto fue deliberadamente simple. Consistía en un menú inicial y en una escena donde el usuario podía crear pequeños elementos conectados entre sí. Al pulsar sobre uno de ellos aparecía una opción para añadir un nuevo elemento debajo, de manera que poco a poco se formaba una estructura parecida a un árbol. Esta prueba estaba todavía lejos de un árbol de decisión real. Su valor estaba en comprobar si la interacción principal de la aplicación tenía sentido.

Para iniciar este proceso, primero hizo falta entender cómo se organiza un proyecto en Godot. Esta base se explica en la Sección 3.1.

5.1.1 Primeras escenas

La primera escena creada fue el menú principal. En ese momento tenía un único botón, ya que su papel era servir como punto de entrada hacia la prueba del árbol. Incluir este menú desde el principio ayudaba a pensar la aplicación como un conjunto de módulos y facilitaba su ampliación posterior. En la Figura 5.1 se ve esta primera versión.

Al entrar en la escena del árbol aparecía un único elemento inicial. Desde él se podían crear nuevos elementos hijos, conectados por líneas y colocados bajo el anterior. También se añadió la posibilidad de borrar partes del árbol, con el fin de comprobar que la estructura podía cambiar durante la ejecución sin que la escena dejase de ser comprensible.

Esta primera prueba tenía una finalidad muy concreta: validar la interacción. Lo importante era comprobar que el usuario podía construir una jerarquía visual, que las conexiones se actualizaban y que la pantalla seguía siendo manejable al crecer el número de elementos. En la Figura 5.2 se muestra el estado de esta escena inicial.

También se dejaron visibles dos botones relacionados con cargar y guardar. En esta versión todavía no tenían una utilidad real. Su presencia servía para reservar un espacio de la interfaz a una necesidad que ya se preveía: poder conservar un árbol y recuperarlo más adelante.

5.1.2 Exportación web con GitHub Pages

Una vez que la prueba interactiva funcionaba dentro de Godot, el siguiente paso lógico fue llevarla al navegador. Esta comprobación era esencial, ya que la aplicación estaba pensada para ser consultada como una página web y también debía poder probarse fuera del editor de desarrollo.

La exportación se realizó correctamente con este proyecto inicial. El resultado se publicó mediante GitHub Pages, usando la configuración del propio repositorio para indicar qué carpeta debía servir como página web. Con esto se confirmó que el flujo completo era viable: desarrollar en Godot, exportar el proyecto y publicarlo en una dirección accesible desde el navegador.

Esta prueba temprana redujo el riesgo del proyecto. Si la exportación web hubiese dado problemas graves, habría sido necesario replantear la tecnología antes de avanzar con los algoritmos. Al funcionar desde el inicio, el desarrollo pudo continuar sobre una base más segura.

5.2 Árbol de decisión: versión 0.2

El proyecto dejó de ser un prototipo aislado cuando comenzó el desarrollo del primer algoritmo: el árbol de decisión. A partir de este punto, el reto pasó a consistir en conseguir que el árbol se construyese siguiendo las decisiones del algoritmo y que el usuario pudiese entender cada paso.

5.2.1 Primera arquitectura del árbol

El primer cambio fue separar la prueba visual inicial de una estructura preparada para crecer. El árbol tenía que convertirse en una vista capaz de representar tanto acciones del

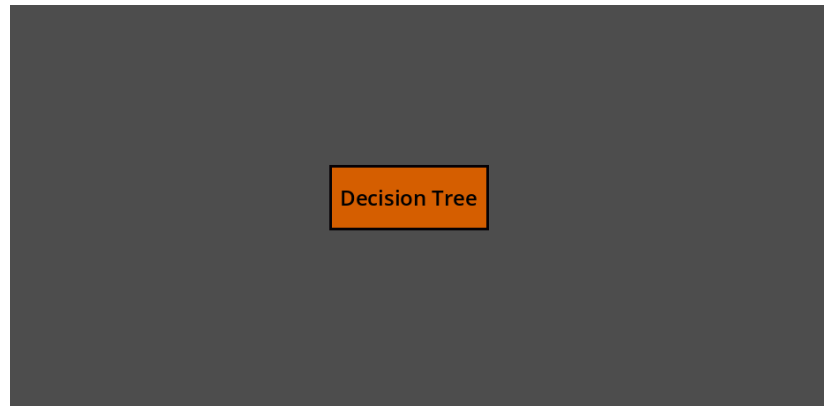


Figura 5.1: Versión preliminar del menú principal.

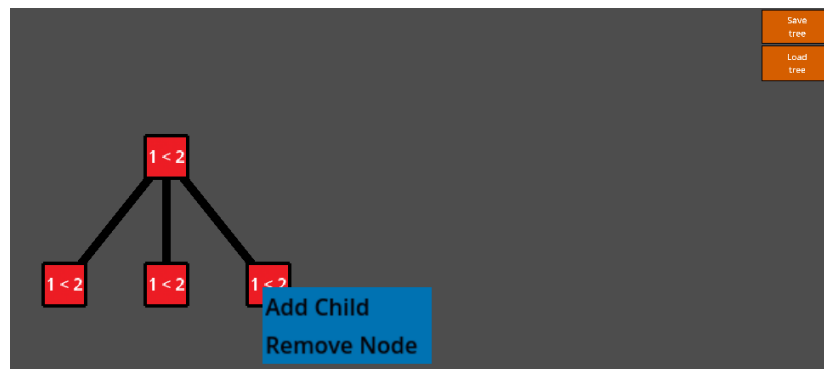


Figura 5.2: Primera escena de prueba del árbol.

usuario como decisiones tomadas por el algoritmo.

Para ello se mantuvo el menú principal como entrada de la aplicación y se creó una vista específica para el árbol. Esta vista debía resolver un problema práctico desde el principio: un árbol puede crecer hacia abajo y hacia los lados con mucha rapidez. Si todo se colocaba en una pantalla fija, una parte del contenido quedaría fuera de la vista en cuanto apareciesen varios niveles. Por esta razón se introdujo una zona desplazable, de forma que el usuario pudiese moverse por el árbol cuando su tamaño aumentase.

También se revisó la forma de colocar los elementos. La versión inicial podía añadir hijos bajo un nodo y el resultado necesitaba más orden. Se adoptó una distribución sencilla: los hijos se colocaban por debajo de su padre y el padre quedaba centrado respecto a ellos. Esta decisión buscaba mantener una lectura clara y estable cada vez que el árbol cambiaba.

Otro aspecto importante fue separar visualmente los elementos del árbol y las líneas que los unen. Esta separación hizo que cada parte tuviese una responsabilidad clara: unos elementos representaban preguntas o resultados, y las líneas representaban el camino seguido entre ellos. Gracias a esto, más adelante fue más sencillo animar las ramas y mostrar sobre ellas el valor que conducía de un punto del árbol al siguiente.

En esta etapa se conservó también el modo manual. El objetivo principal era construir árboles de decisión, y el modo manual seguía siendo útil para probar la interfaz, comprobar el desplazamiento por la pantalla y detectar problemas de colocación sin depender todavía del algoritmo.

El siguiente avance fue preparar el modo automático. En este modo, el usuario dejaba de añadir ramas a mano y pasaba a controlar el avance del entrenamiento. La primera versión de esta interfaz era todavía muy simple, con botones en inglés, como se muestra en la Figura 5.3. Esta pantalla introducía una idea que se mantuvo durante todo el desarrollo: el algoritmo avanzaría al ritmo marcado por el usuario.

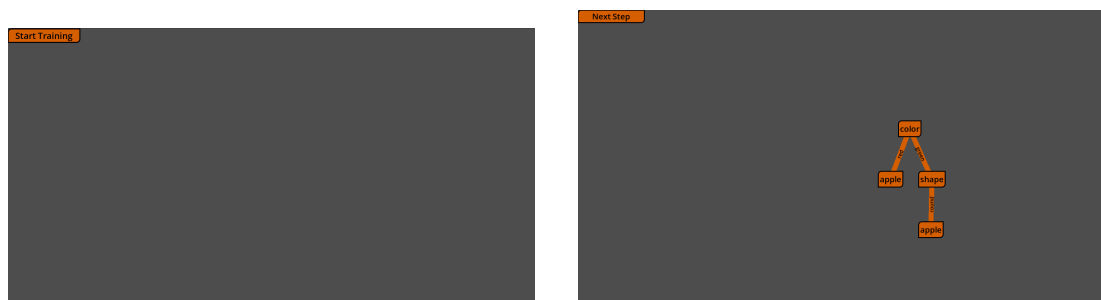


Figura 5.3: Primer menú de control del entrenamiento algorítmico, con los botones para iniciar el entrenamiento y avanzar al siguiente paso antes de traducir la interfaz y aplicar los temas visuales definitivos.

Para conseguirlo, el algoritmo se organizó de forma incremental. La construcción com-

pleta se repartía en pasos pequeños: se guardaba el punto en el que se encontraba el proceso y se resolvía una parte cada vez que el usuario pulsaba el botón de avanzar. Esta decisión fue clave para el propósito educativo de la aplicación, porque permitía observar cómo se forma el árbol paso a paso.

La primera prueba de entrenamiento utilizó un conjunto de datos pequeño y controlado. Su objetivo era comprobar que el árbol se creaba correctamente, que las ramas aparecían donde debían y que la interfaz se actualizaba después de cada paso. En esta fase, lo más difícil fue coordinar tres niveles al mismo tiempo: el estado interno del algoritmo, la estructura del árbol y lo que el usuario veía en pantalla.

5.2.2 Reorganización de la escena y primeros elementos explicativos

Cuando el modo automático empezó a funcionar, se reorganizó la escena para simplificarla. Algunas piezas que habían sido útiles durante las pruebas iniciales dejaron de ser necesarias, así que se redujo la cantidad de elementos intermedios. Esta limpieza facilitó seguir avanzando sin arrastrar soluciones provisionales.

Uno de los problemas que apareció al reorganizar la vista fue el centrado del árbol. Al crecer, el árbol podía quedar desplazado hacia una esquina, lo que dificultaba seguir la construcción. Se ajustó entonces el comportamiento de la zona desplazable para que el contenido apareciese centrado y el usuario pudiese moverse por él de forma natural. Esta mejora mantenía intacto el algoritmo y afectaba mucho a la experiencia de uso.

La escena del árbol empezó a funcionar como un pequeño flujo guiado. Primero se mostraba un panel para escoger entre crear el árbol manualmente o dejar que lo construyese el algoritmo. Después, según la opción elegida, aparecían los controles correspondientes. En la Figura 5.4 se muestra este panel inicial.

En esta misma etapa se empezó a cuidar más la apariencia de los nodos. Los puntos que representaban una decisión y los puntos que representaban una clasificación final pasaron a tener estilos diferentes. Esta distinción visual ayudaba a leer el árbol sin depender tanto del texto: algunas partes indicaban que todavía había una pregunta que responder, y otras indicaban que se había llegado a una conclusión.

También se tradujo la interfaz al castellano. Los botones iniciales pasaron a llamarse “Empezar entrenamiento” y “Siguiente paso”. Este cambio era pequeño desde el punto de vista técnico y resultaba importante para que la aplicación fuese coherente con el resto del proyecto. En la Figura 5.5 se ve la interfaz después de esta adaptación y de los primeros cambios visuales.

El siguiente paso natural fue añadir una ventana informativa. Hasta ese momento, el árbol mostraba qué se estaba creando y apenas explicaba por qué se tomaba cada decisión. La ventana nació para cubrir esa carencia: debía acompañar a la construcción del árbol con una



Figura 5.4: Panel inicial de la escena del árbol para escoger entre creación manual y creación algorítmica.

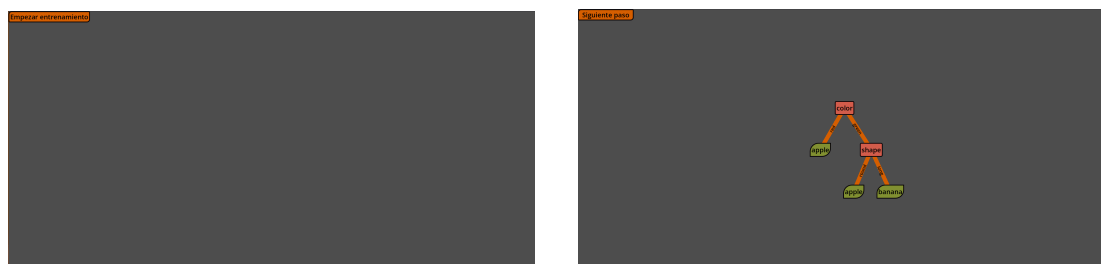


Figura 5.5: Interfaz de entrenamiento del árbol tras traducir los controles. A la izquierda se muestra el botón para comenzar el entrenamiento y a la derecha el estado del árbol al avanzar con el botón de siguiente paso, con temas distintos para nodos internos y hojas.

explicación escrita del paso actual.

Su primera versión contenía un mensaje inicial que invitaba al usuario a comenzar el entrenamiento. Más adelante se convirtió en un componente propio, separado de la vista principal, porque iba a necesitar textos distintos según el caso y debía poder actualizarse cada vez que avanzase el algoritmo. En la Figura 5.6 se puede ver esta primera integración.

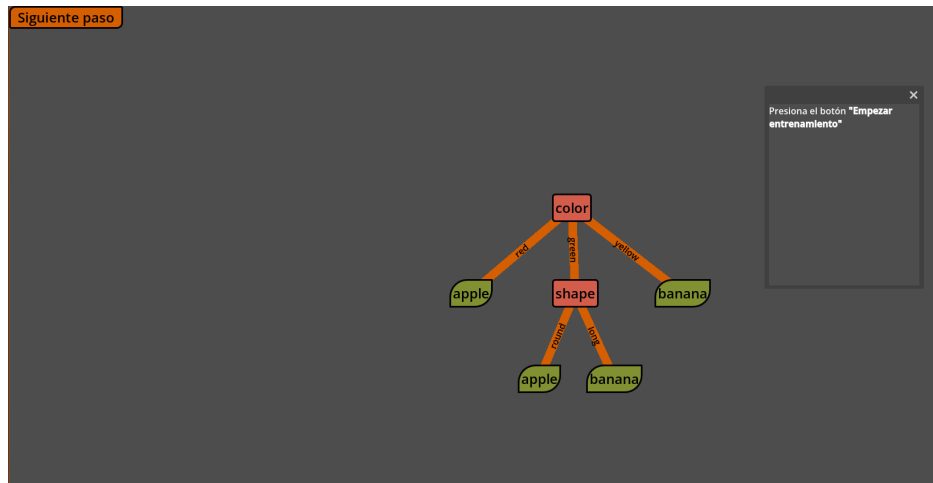


Figura 5.6: Primera versión de la ventana informativa integrada en la vista del árbol, todavía con un mensaje inicial que indicaba al usuario que debía comenzar el entrenamiento.

Los primeros textos de la ventana explicaban tres situaciones frecuentes. La primera era la creación de un nodo de decisión, cuando el algoritmo todavía tenía que escoger por qué característica dividir los datos. La segunda era la creación de una hoja porque todos los ejemplos que llegaban a ese punto tenían la misma respuesta. La tercera era la creación de una hoja por mayoría, usada cuando ya no había más preguntas útiles y se debía escoger la respuesta más frecuente.

Durante estas pruebas se detectó que el texto necesitaba una preparación previa. Algunas listas aparecían con un formato propio de programación y poco adecuado para la lectura. Por ello se preparó la información antes de enseñarla, transformándola en frases más naturales. Esta fue una decisión importante para el tono de la aplicación: la ventana debía parecer una explicación dirigida al usuario.

También se empezó a mostrar el criterio usado para escoger la siguiente división. En esta primera fase aparecía como texto, con una línea por cada opción evaluada. Era una solución básica y permitió conectar el cálculo del algoritmo con una explicación visible dentro de la interfaz. A partir de aquí, la aplicación construía el árbol y empezaba a justificarlo.

5.2.3 Animación de la construcción del árbol

Con la construcción paso a paso funcionando, apareció un nuevo problema: los cambios eran demasiado bruscos. Cada pulsación podía crear un nodo, mover partes del árbol y dibujar nuevas ramas. El resultado era correcto. En algunos pasos, el usuario podía perder de vista qué elemento acababa de aparecer.

Para mejorar esta lectura se añadieron animaciones. Primero se animaron las ramas, de manera que cada conexión parecía crecer desde el punto anterior hasta el nuevo. Después se animó la aparición de los nodos, que dejaban de mostrarse de golpe y pasaban a aparecer de forma gradual. Estas animaciones dirigían la atención hacia el cambio producido en cada paso.

Al introducir animaciones fue necesario revisar cómo se actualizaban las conexiones. En las primeras versiones, al modificar el árbol se redibujaban todas las ramas. Esto hacía que, cada vez que aparecía una conexión nueva, también se reiniciasen las que ya existían. Se cambió este comportamiento para conservar las ramas anteriores y animar solo las recién creadas. Gracias a ello, el avance resultaba más claro y menos repetitivo.

Después se incorporó una idea pensada específicamente para explicar mejor el algoritmo: los nodos pendientes. Cuando el árbol decide dividir los datos por una característica, ya se sabe qué caminos van a aparecer y todavía falta conocer qué habrá al final de cada uno. Para representar ese estado intermedio, la aplicación empezó a mostrar nodos temporales con un aspecto distinto.

Estos nodos pendientes indicaban que el algoritmo ya había abierto una rama y que aún quedaba por resolver su contenido. Más adelante, cuando esa parte del árbol se procesaba, el nodo temporal se transformaba en una decisión real o en una clasificación final. Esta transición también se animó para que el usuario percibiese el cambio y entendiese que se estaba completando una parte que antes solo estaba prevista.

Otra prueba temprana consistió en mostrar un valor numérico al pasar el cursor por ciertos nodos. La intención era permitir consultar información sobre la decisión tomada sin llenar todo el árbol de números. La idea funcionaba de forma limitada: un número aislado dice poco si no se explica qué significa. Por esa razón, más adelante esa información se trasladó hacia la ventana informativa, donde podía acompañarse de una explicación más completa.

5.2.4 Gráficas y casos explicativos en la ventana

La ventana informativa evolucionó después de los primeros textos. A medida que el árbol crecía y los pasos se volvían más ricos, una única explicación en texto plano resultaba insuficiente. El objetivo pasó a ser combinar texto y apoyo visual para que la decisión del algoritmo fuese más fácil de seguir.

El cambio principal fue introducir una gráfica de barras. La ventana pasó de una lista de valores a una comparación visual entre las opciones que el algoritmo estaba evaluando. Así era más sencillo ver cuál destacaba y por qué se escogía una división determinada.

Para integrar esta gráfica, la ventana pasó a tener una estructura desplazable. Esto era necesario porque algunas explicaciones ocupaban más espacio que otras. El usuario podía leer la primera parte del texto, observar la gráfica y continuar después con la explicación del resultado. De esta forma, la gráfica quedaba dentro del relato del paso actual.

Este cambio obligó también a preparar mejor la información que recibía la ventana. Los valores utilizados para la gráfica debían conservarse como datos numéricos, sin transformarlos desde el inicio en frases ya formateadas. Esta separación permitió que la misma información sirviese para explicar el paso con texto y para construir la representación visual.

En esta etapa se añadió además un caso explicativo para los empates. Durante las pruebas apareció una situación en la que varias opciones podían ser igual de buenas. Si la aplicación escogía una sin explicar nada, la decisión podía parecer arbitraria. Por eso se incorporó un texto específico que indicaba que había varias alternativas equivalentes y que el algoritmo continuaba con una de ellas.

También se revisaron los textos de las hojas. Las primeras frases estaban escritas pensando en ejemplos muy pequeños, donde a veces solo llegaba un dato a cada resultado final. Al probar conjuntos más variados, esa redacción dejó de ser válida. Se sustituyó por una explicación más general, capaz de describir tanto un único ejemplo como varios ejemplos que terminan en la misma clasificación.

Este proceso se repitió varias veces durante el desarrollo: al probar nuevos datos aparecían casos que la explicación inicial no cubría bien. Esos casos se fueron incorporando al discurso de la aplicación para que el usuario no tuviese que interpretar silencios o comportamientos inesperados.

5.2.5 Primer panel de selección de datos

Hasta este momento, los datos usados para entrenar el árbol estaban fijados dentro del propio proyecto. Esto era cómodo para las primeras pruebas, porque permitía centrarse en el funcionamiento del algoritmo y de la interfaz. Con el tiempo, esta comodidad empezó a limitar la aplicación: el usuario necesitaba poder escoger con qué ejemplo trabajar.

El siguiente paso fue separar la elección de los datos del entrenamiento. Primero se probó un segundo conjunto preparado manualmente, con casos diseñados para forzar ramas más profundas y situaciones menos evidentes. Esta prueba ayudó a comprobar que el árbol también funcionaba con ejemplos distintos del primero y que los textos seguían teniendo sentido cuando los datos cambiaban.

A partir de ahí se creó un panel de selección. Su papel era guiar al usuario antes de empezar

el entrenamiento: primero se elegía una fuente de datos y después se iniciaba la construcción del árbol. La primera opción permitía usar el conjunto de ejemplo incluido en la aplicación. La segunda opción abría la puerta a cargar un archivo CSV.

La carga de archivos se introdujo de forma progresiva. En una primera versión, el panel ya mostraba la opción de CSV y todavía estaba incompleta. Después se añadió la lectura del archivo y la comprobación de que la selección fuese válida antes de permitir entrenar. También se añadieron mensajes de éxito o error para que el usuario supiese si podía continuar.

Con este panel, el flujo del modo algorítmico cambió de manera importante. Antes, el usuario pulsaba directamente el botón de comenzar. Después de esta mejora, primero escogía los datos, luego confirmaba la selección y finalmente avanzaba por la construcción del árbol. Esta separación hacía el proceso más comprensible, porque dejaba claro que el entrenamiento depende de los datos elegidos.

La versión inicial del panel todavía era provisional. Quedaban detalles por ajustar, como mejorar la detección de la respuesta esperada en archivos externos y pulir la transición entre la selección de datos, el árbol y la ventana informativa. Con esas limitaciones pendientes, el cambio marcó un avance importante: el árbol dejó de estar ligado a un único ejemplo y empezó a comportarse como una herramienta más flexible.

5.2.6 Carga de archivos y navegación por pasos

El primer ajuste tras introducir el panel de selección fue hacer que los archivos CSV pudiesen usarse realmente para entrenar el árbol. Hasta ese momento, la carga de datos era una posibilidad visible en la interfaz y todavía necesitaba más trabajo para integrarse en el flujo completo. El problema principal estaba en distinguir qué parte del archivo describía cada ejemplo y qué parte contenía la respuesta que el árbol debía aprender.

Para resolverlo, la aplicación empezó a buscar automáticamente la columna que representaba esa respuesta. La solución se planteó de forma práctica: se reconocían algunos nombres habituales y se separaban del resto de características. Con ello, el usuario podía cargar archivos sencillos sin tener que modificar manualmente la estructura de los datos antes de entrenar.

También se revisó la lectura del archivo para hacerla más fiable. El objetivo era aceptar tablas simples, con una primera fila de nombres y varias filas de ejemplos, ignorando las líneas vacías. Esta mejora seguía siendo limitada, ya que estaba pensada para datos formados por categorías y con una estructura clara. Con esa base bastaba para probar el árbol con ejemplos externos al proyecto.

El flujo visual se ajustó a esta nueva forma de empezar. Al escoger el modo algorítmico, primero aparecía el panel para seleccionar los datos. El árbol y la ventana informativa permanecían ocultos hasta que el usuario hacía una selección válida. Así se evitaba mostrar una escena vacía o una explicación sin contexto. Una vez elegidos los datos, la vista pasaba di-

rectamente al entrenamiento y la ventana indicaba que el siguiente paso consistía en avanzar por la construcción.

En paralelo empezó a prepararse la navegación hacia atrás. Hasta ese momento, el entrenamiento solo permitía avanzar. Esta limitación era poco adecuada para una herramienta educativa, porque el usuario podía querer revisar justo el paso anterior para entender por qué había aparecido una rama o una hoja. Por eso se añadió la idea de un botón de paso previo.

La primera versión de esta navegación hacia atrás solo preparaba el camino. La aplicación empezó a recordar qué pasos ya se habían ejecutado y cuáles quedaban pendientes, de forma que más adelante pudiera deshacer una decisión concreta. Después se añadió el botón “Paso previo” a la interfaz. Con él, la construcción del árbol dejó de ser estrictamente lineal: el usuario podía avanzar con “Siguiente paso” y retroceder cuando necesitase revisar el proceso.

Hacer que el retroceso funcionase de verdad requirió pensar qué debía ocurrir con cada tipo de elemento del árbol. Si se deshacía una decisión que había abierto varias ramas, también había que retirar las partes que dependían de ella. Si el elemento deshecho era temporal, resultaba más claro devolverlo a su estado pendiente que eliminarlo por completo. Si se volvía al inicio, el árbol debía quedar vacío y sin conexiones antiguas en pantalla.

Durante las pruebas apareció precisamente ese último problema. Al regresar al primer paso, algunas conexiones podían permanecer visibles después de eliminar el árbol. La corrección consistió en actualizar también las ramas cuando el árbol quedaba vacío. Con esto, retroceder hasta el inicio dejaba la vista en un estado coherente y permitía avanzar de nuevo sin arrastrar restos visuales de la ejecución anterior.

5.2.7 Primera evaluación visual del árbol

El último avance de este tramo inicial fue una primera versión del modo de evaluación. Hasta entonces, la aplicación había mostrado cómo se construía el árbol. Todavía faltaba enseñar cómo se usa ese árbol para clasificar ejemplos nuevos. La evaluación se planteó de forma visual: cada ejemplo de prueba se representaría como un objeto que atraviesa el árbol siguiendo las decisiones que ya se habían aprendido.

Para introducir este modo se añadieron controles específicos a la interfaz. Durante el entrenamiento, la evaluación permanecía desactivada. Al terminar la construcción del árbol, el usuario podía pasar a una nueva fase y escoger datos para probar el resultado. Se reutilizó el panel de selección, ya que su papel seguía siendo el mismo: elegir con qué datos se iba a trabajar.

La aplicación empezó entonces a distinguir entre dos momentos. En el primero, los datos seleccionados servían para construir el árbol. En el segundo, los datos seleccionados servían para comprobar cómo clasificaba el árbol ya entrenado. Esta separación todavía era provisional y permitía probar el ciclo completo: entrenar primero y evaluar después.

Para que la evaluación se entendiese mejor, cada ejemplo se dibujó como una pieza sencilla que podía moverse por la escena. Al pulsar el botón correspondiente, una de esas piezas entraba en el árbol y se desplazaba desde la raíz hasta una hoja. En cada punto seguía la rama que coincidía con sus características, hasta llegar a la clasificación final.

Esta primera versión era básica. Los ejemplos se representaban con formas simples y cada recorrido se ejecutaba de una vez hasta el final. Las mejoras posteriores, como avanzar nivel a nivel, agrupar los ejemplos bajo las hojas o explicar cada decisión en la ventana informativa, quedaron para una fase posterior. Esta prueba dejó establecida la idea central del modo de evaluación: ver físicamente cómo un ejemplo recorre las decisiones del árbol hasta llegar a una clasificación.

Con este conjunto de cambios quedó completada la primera versión significativa del árbol de decisión. El proyecto pasó de dibujar elementos conectados a construir un árbol paso a paso, mostrar estados pendientes, animar la aparición de ramas y nodos, explicar las decisiones con texto y gráficos, cargar datos sencillos y ejecutar una primera evaluación visual. A partir de aquí, el desarrollo continuó con mejoras de interfaz, persistencia, adaptación a móvil y explicaciones más completas, que se tratarán en apartados posteriores para separar esta primera fase de las revisiones finales.

Conclusiones

DERRADEIRO capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras,...

Capítulo 7

Trabajo futuro

Apéndices

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque.

Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliografía

- [1] G. Sanderson, “Neural networks,” 3Blue1Brown, consultado el 10 de junio de 2026. [En línea]. Disponible en: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- [2] —, “But what is a neural network?” 3Blue1Brown, consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://www.3blue1brown.com/?topic=neural-networks>
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [4] D. Smilkov and S. Carter, “A neural network playground,” TensorFlow, consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://playground.tensorflow.org/>
- [5] M. Kahng, N. Thorat, P. Chau, F. Viégas, and M. Wattenberg, “Gan lab: Play with generative adversarial networks in your browser!” Georgia Tech and Google Brain/PAIR, consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://poloclub.github.io/ganlab/>
- [6] Interactive ML, “Interactive decision trees learning tool,” consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://www.interactive-ml.com/decision-tree.html>
- [7] Godot Engine contributors, “Godot engine,” consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://godotengine.org/>
- [8] —, “Godot documentation,” consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://docs.godotengine.org/>
- [9] —, “Exporting for the web,” consultado el 10 de junio de 2026. [En línea]. Disponible en: https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_web.html
- [10] Python Software Foundation, “Python,” consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://www.python.org/>

- [11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. VanderPlas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [12] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshain, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [13] The Rust Project Developers, “Rust programming language,” consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://www.rust-lang.org/>
- [14] ratex contributors, “ratex,” crates.io, consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://crates.io/crates/ratex>
- [15] World Wide Web Consortium, “Webassembly core specification,” consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://www.w3.org/TR/wasm-core-2/>
- [16] Mozilla Developer Network, “Javascript,” consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [17] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014, consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://git-scm.com/book/en/v2>
- [18] GitHub, Inc., “Github pages documentation,” consultado el 10 de junio de 2026. [En línea]. Disponible en: <https://docs.github.com/en/pages>